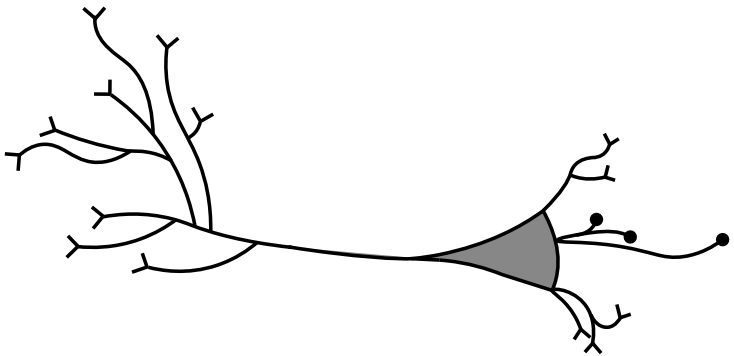
# Neural coding and population representations

## Wits Computational Neuroscience COMS4070A / COMS7081A: Lecture 4

August 12, 2026

Kira M. Düsterwald



kira.dusterwald.21@ucl.ac.uk

# Where we are and where we are going



**Lectures 1-3: Core properties of (single) neurons**

- *Sub-single neuron*: Ion channels, Hodgkin–Huxley, action potential
- *'Point' neuron*: Dynamical systems, leaky integrate-and-fire (LIF)
- *How other neurons affect a single neuron*: Learning, synaptic weights, STDP

# Where we are and where we are going



**Lectures 1-3: Core properties of (single) neurons**

- *Sub-single neuron*: Ion channels, Hodgkin–Huxley, action potential
- *'Point' neuron*: Dynamical systems, leaky integrate-and-fire (LIF)
- *How other neurons affect a single neuron*: Learning, synaptic weights, STDP

Lectures 4-6: How do (groups of) neurons *code* relevant information?

- Neural coding and tuning curves
- Spike-triggered average (STA)
- Generalised linear models (GLMs)
- Latent variable models (LVMs)

# Where we are and where we are going



**Lectures 1-3: Core properties of (single) neurons**

- *Sub-single neuron*: Ion channels, Hodgkin–Huxley, action potential
- *'Point' neuron*: Dynamical systems, leaky integrate-and-fire (LIF)
- *How other neurons affect a single neuron*: Learning, synaptic weights, STDP

Lectures 4-6: How do (groups of) neurons *code* relevant information?

- Neural coding and tuning curves
- Spike-triggered average (STA)
- Generalised linear models (GLMs)
- Latent variable models (LVMs)
- *Later: Decision-making, dynamic reps (E-I balance), memory*
- *Applications: Perception, psychiatry, high-dimensional neural data analysis, bio-inspired computer algorithms...*

# Where we are and where we are going



**Lectures 1-3: Core properties of (single) neurons**

- *Sub-single neuron*: Ion channels, Hodgkin–Huxley, action potential
- *'Point' neuron*: Dynamical systems, leaky integrate-and-fire (LIF)
- *How other neurons affect a single neuron*: Learning, synaptic weights, STDP

*How* is neural code implemented?

Lectures 4-6: How do (groups of) neurons *code* relevant information?

- Neural coding and tuning curves
- Spike-triggered average (STA)
- Generalised linear models (GLMs)
- Latent variable models (LVMs)
- *Later: Decision-making, dynamic reps (E-I balance), memory*
- *Applications: Perception, psychiatry, high-dimensional neural data analysis, bio-inspired computer algorithms...*

*What* is the neural code and *why* does it take the form it does?

# What do neurons do?



So we have learned how to model a neuron... But *why* do we care?

# What do neurons do?



So we have learned how to model a neuron... But *why* do we care?

**Neuronal activity represents sensory coding and behaviour**



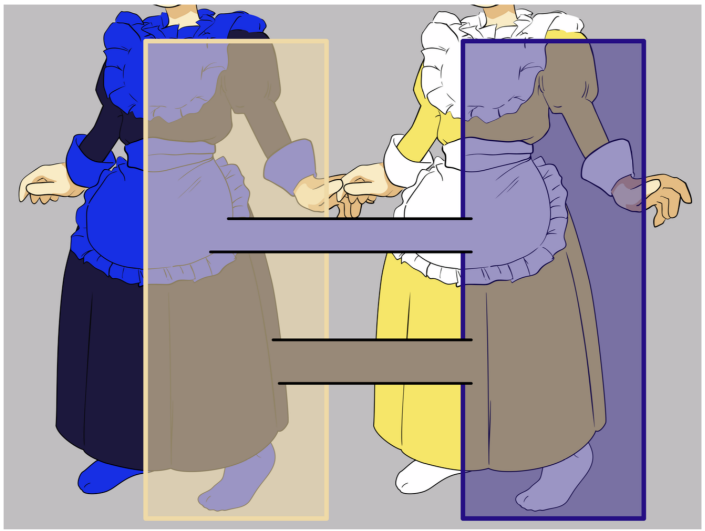
# What do neurons do?



So we have learned how to model a neuron... But *why* do we care?

**Neuronal activity represents sensory coding and behaviour**

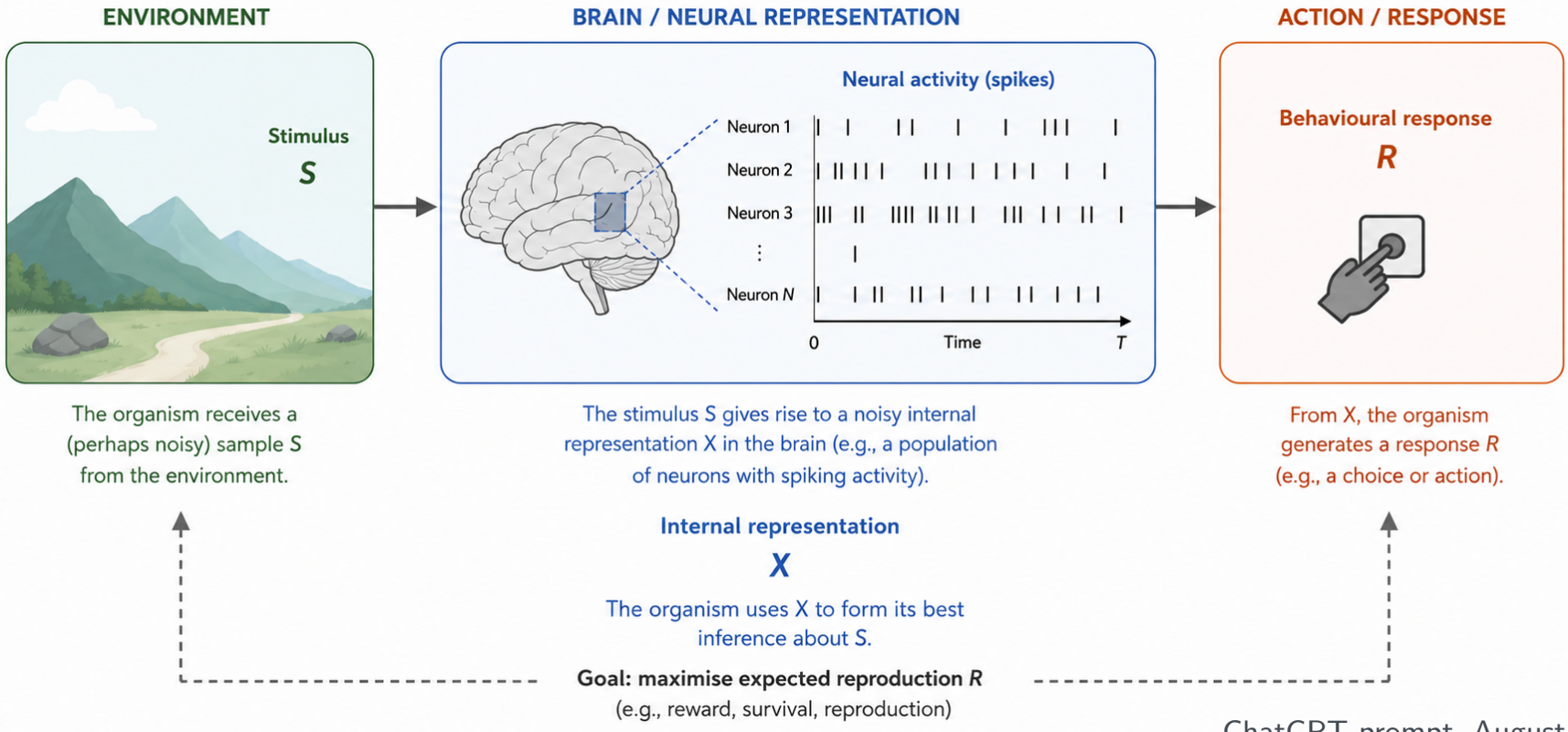




Kasuga jawiki; CC BY-SA 3.0: Wikimedia Commons

# Stimulus $S \rightarrow$ Neural representation $X \rightarrow$ Response $R$

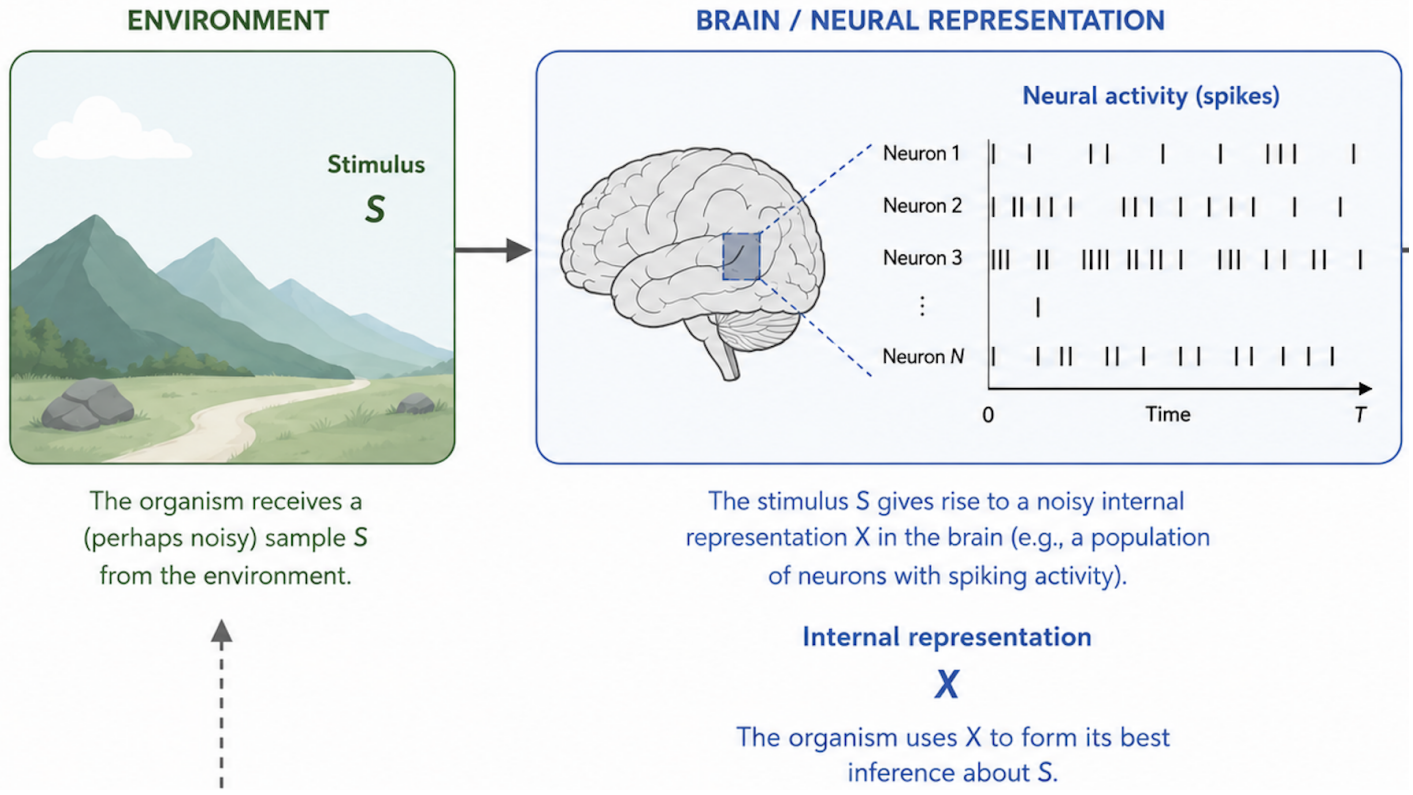





ChatGPT prompt, August 2026

# Stimulus $S \rightarrow$ Neural representation $X \rightarrow$ Response $R$





# Stimulus $S \rightarrow$ Neural representation $X \rightarrow$ Response $R$



WITS UNIVERSITY

A lot of neuroscience is centred around coming up with something we know ought to be represented in the brain and showing it has a neural correlate — which works surprisingly well.

# Stimulus $S \rightarrow$ Neural representation $X \rightarrow$ Response $R$



WITS UNIVERSITY

A lot of neuroscience is centred around coming up with something we know ought to be represented in the brain and showing it has a neural correlate. What correlates could be proxies for $X$?

- Membrane potential
- Spikes – as **firing rates** or with spike timing dependence
- Single vs. population activity
- Cell-type specific sub-cellular ion fluxes
- Other cells like astrocytes

# Stimulus $S \rightarrow$ Neural representation $X \rightarrow$ Response $R$



WITS UNIVERSITY

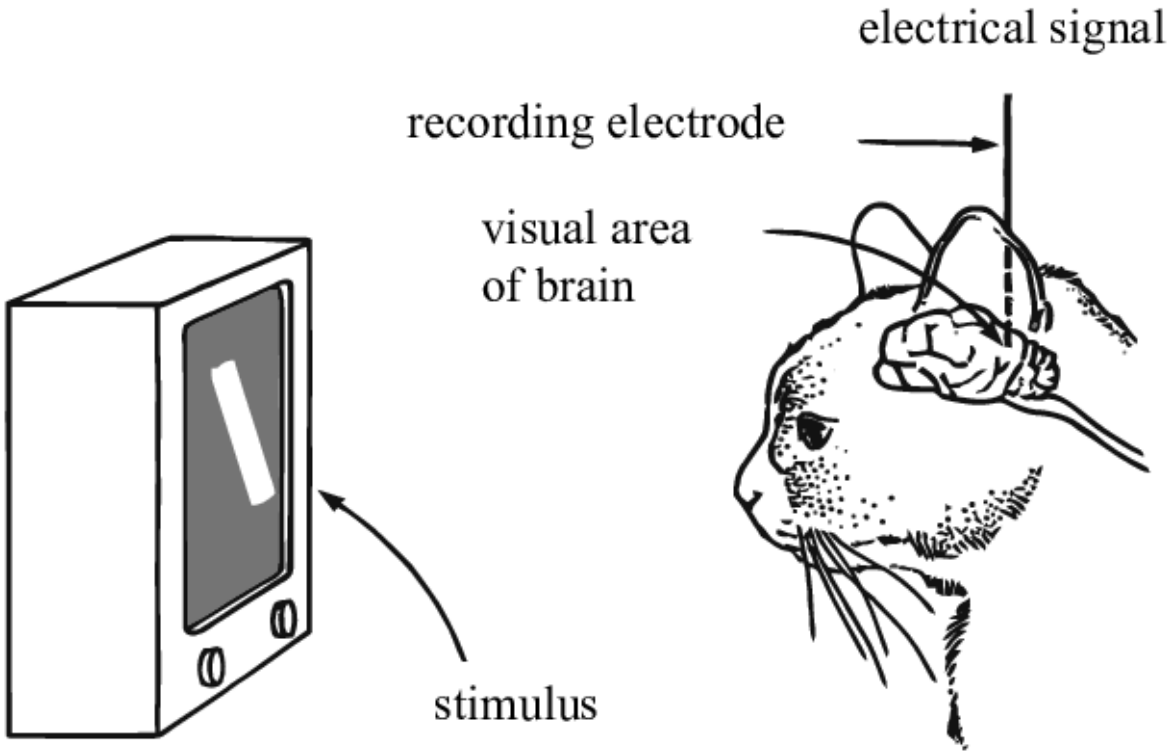
What correlates could be proxies for $X$?

- Membrane potential
- Spikes – as **firing rates** or with spike timing dependence
- Single vs. population activity
- Cell-type specific sub-cellular ion fluxes
- Other cells like astrocytes

↑ *as measured by electrophysiology (like patch clamps), local field potentials (like neuropixel probes or microelectrode arrays), calcium imaging of calcium flux after action potentials, wider field activity (via EEG,or fMRI), and many other techniques!*

# Example: Orientation-selectivity

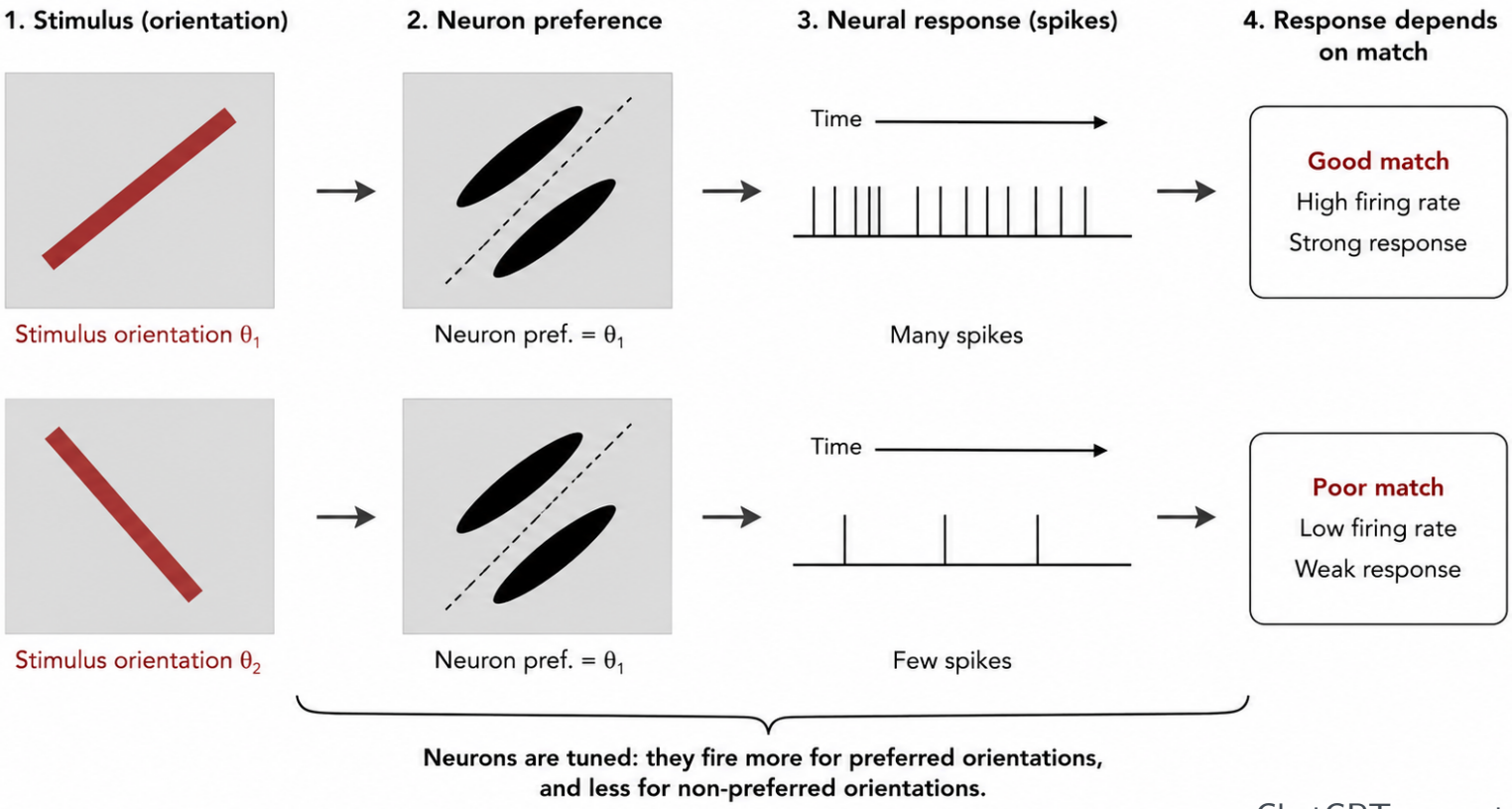





Hubel and Weisel, 1959; Nguyen et al., 2019

# Example: Orientation-selectivity



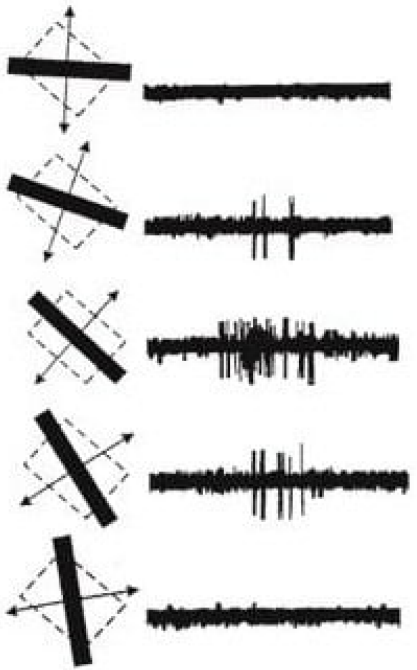


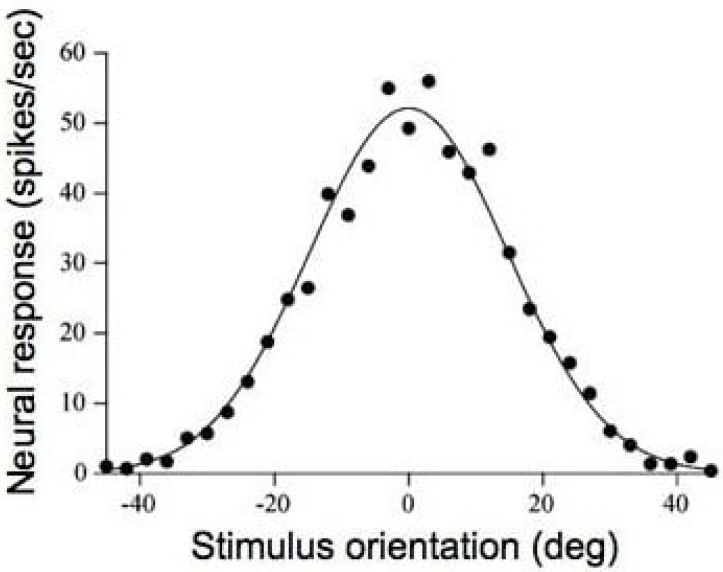

ChatGPT prompt, August 2026

# Example: Orientation-selectivity



Tuning curves are the single-neuron representation of the $S \to X$ code!

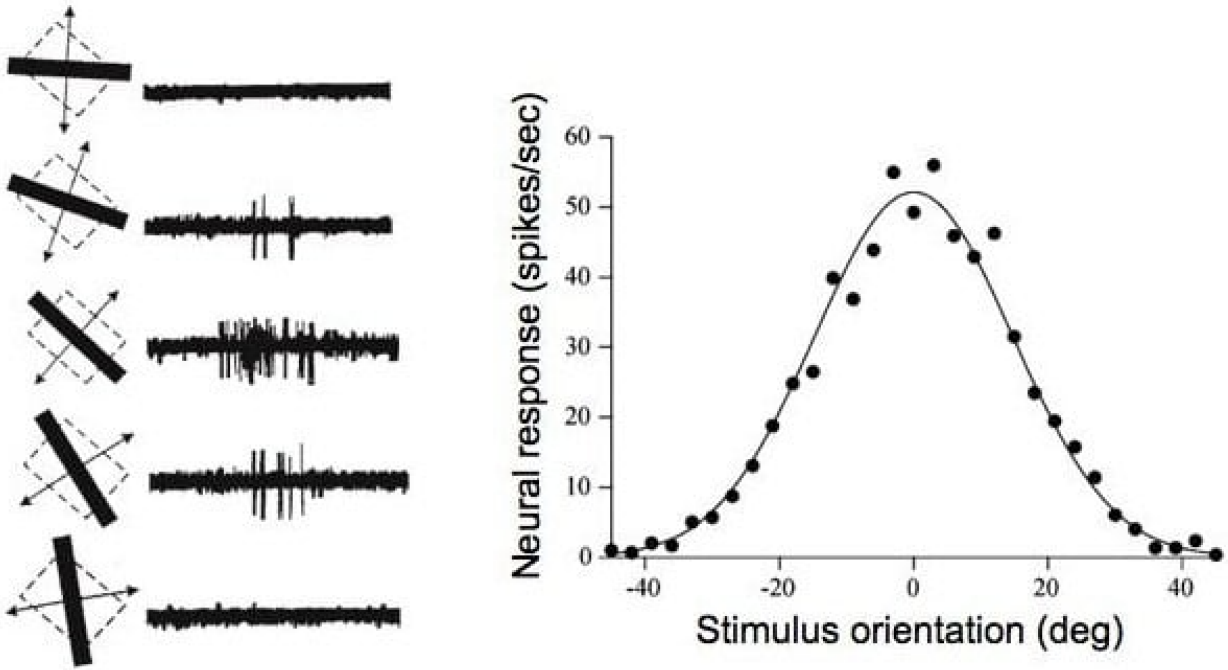





Hubel and Weisel, 1968

# Example: Orientation-selectivity

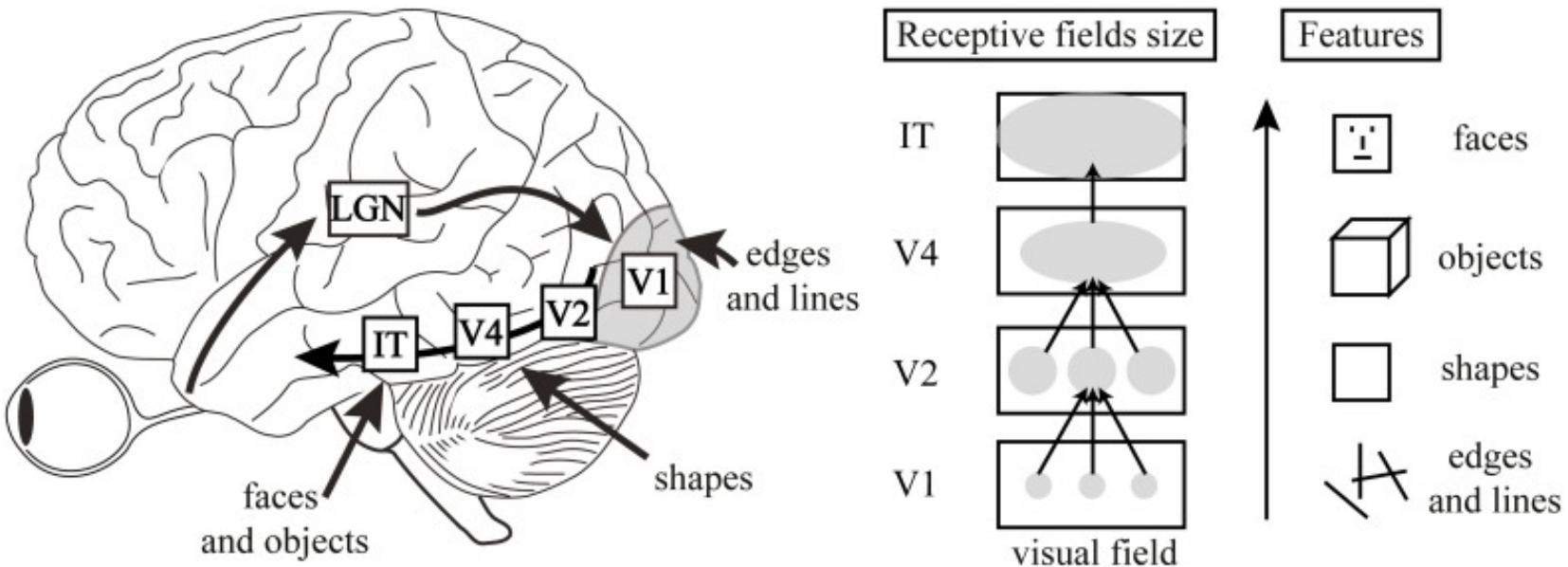
Tuning curves are the single-neuron representation of the $S \to X$ code!




$\Longrightarrow$ a cell's preferred stimulus, tuning width, baseline response and gain! Hubel and Weisel, 1968

# Example: Orientation-selectivity

The concept of tuning curve and selectivity generalises along the neural hierarchy and to other features (e.g. in V1: colour, spatial frequency, ocular dominance, ...).




Herzog and Clarke, 2014

*Note: higher-order features do not in general just arise from linear combinations of simple features — more on this later.*

# How do we find what a neuron is tuned to?



Suppose we don't know the neuron's tuning curve, but we observe the stimulus and the neuron's spikes.




*Which stimulus precedes the spikes?*

# How do we find what a neuron is tuned to?

Suppose we don't know the neuron's tuning curve, but we observe the stimulus and the neuron's spikes.




*Which stimulus precedes the spikes?*

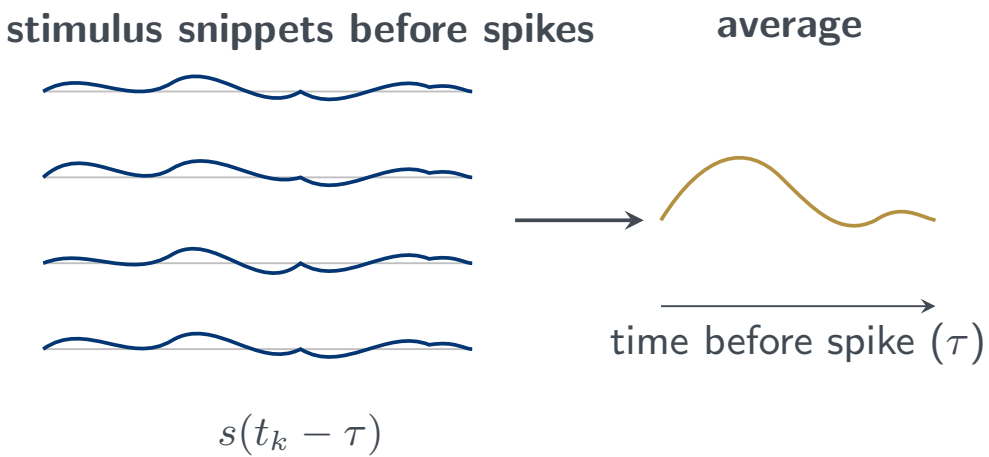
Ans: Align the stimulus to each spike and average: **spike-triggered average.**

# The spike-triggered average (STA)

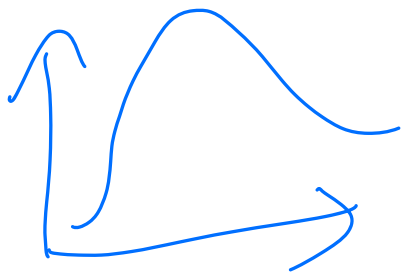


Suppose spikes occur at times $t_1, t_2, \ldots, t_N$.

For each spike, extract the preceding stimulus:




$$\mathrm{STA}(\tau) = \frac{1}{N} \sum_{k=1}^{N} s(t_k - \tau)$$



# Why does STA work? (Decoding)

**uncorrelated stimuli**
average away



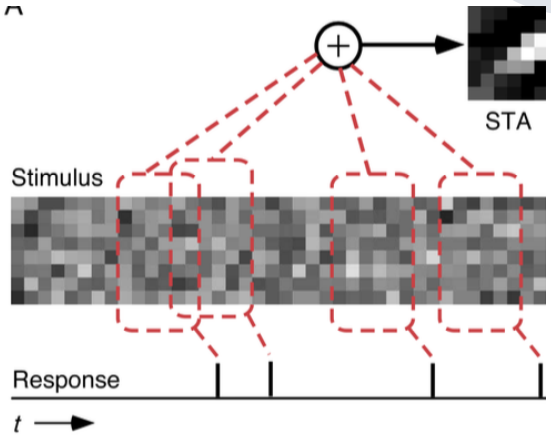
$\approx 0$

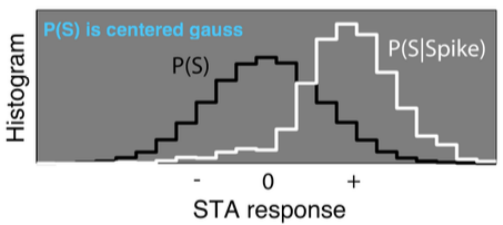
**consistent stimulus features**
survive averaging



**STA** finds them





# From STA to modelling firing rate: Encoding



The spike-triggered average can be used as a **linear filter** to predict the neural response.

$$\hat{r}(t) = \int_0^T s(t-\tau)\, w(\tau)\, d\tau$$

where

- $s(t)$ is the stimulus
- $w(\tau)$ is the STA filter
- $T$ is the stimulus history window
- $\hat{r}(t)$ is the predicted response

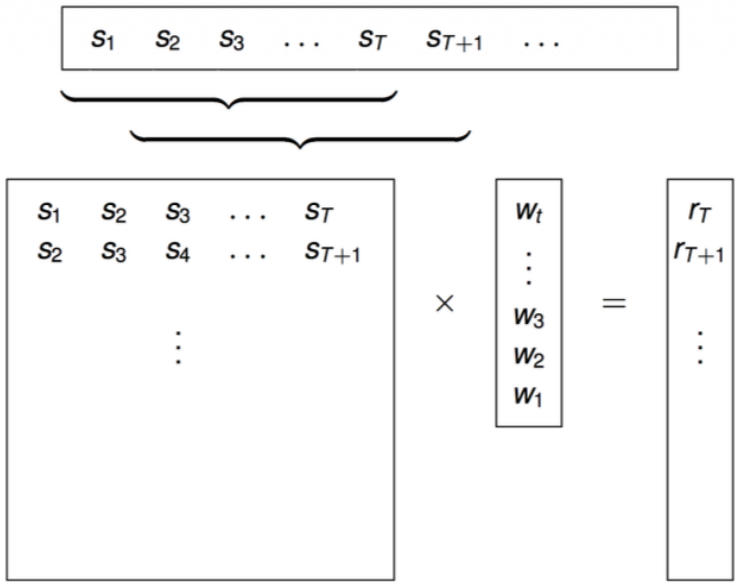


# Estimating the STA filter: Encoding

For discrete time points, construct a design matrix from shifted stimulus windows $S$. Then

$$\hat{\mathbf{r}} = S\mathbf{w}$$

where $\mathbf{w}$ contains the filter coefficients.



# Estimating the STA filter: Encoding



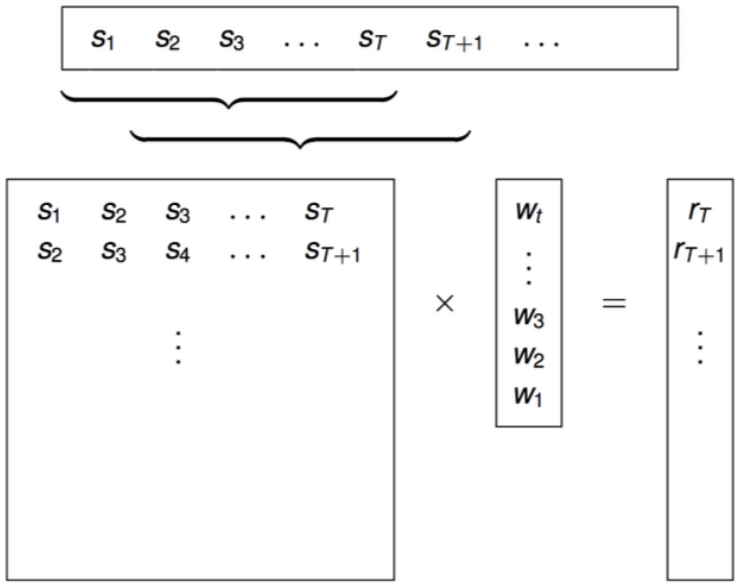
For discrete time points, construct a design matrix from shifted stimulus windows $S$. Then

$$\hat{\mathbf{r}} = S\mathbf{w}$$

where $\mathbf{w}$ contains the filter coefficients.

We can solve this with linear regression!

$$\boxed{\hat{\mathbf{w}} = (S^\top S)^{-1} S^\top \mathbf{r}}$$



# Estimating the STA filter: Encoding

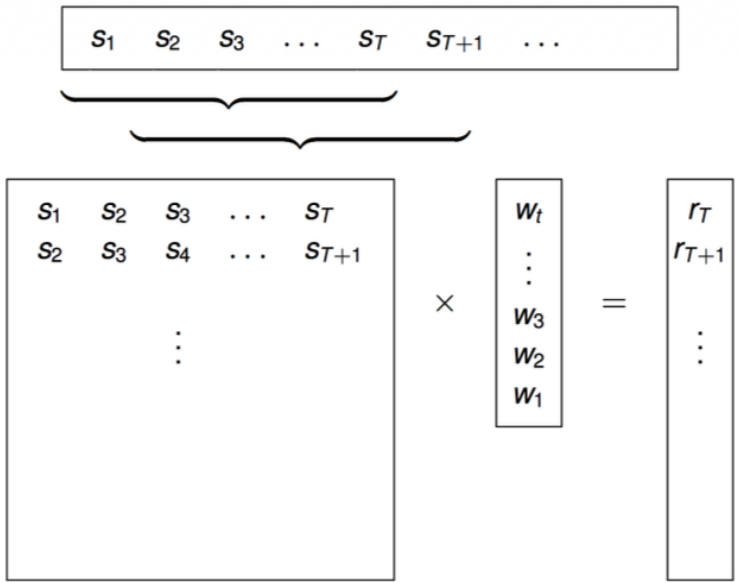
For discrete time points, construct a design matrix from shifted stimulus windows $S$. Then

$$\hat{\mathbf{r}} = S\mathbf{w}$$

where $\mathbf{w}$ contains the filter coefficients.

We can solve this with linear regression!

$$\boxed{\hat{\mathbf{w}} = (S^\top S)^{-1} S^\top \mathbf{r}}$$




*Aside: For Gaussian white-noise stimuli, the STA recovers the linear filter $\mathbf{w}$ up to scale (Bussgang's Theorem).*

# Estimating the STA filter: Encoding





For discrete time points, construct a design matrix from shifted stimulus windows $S$. Then

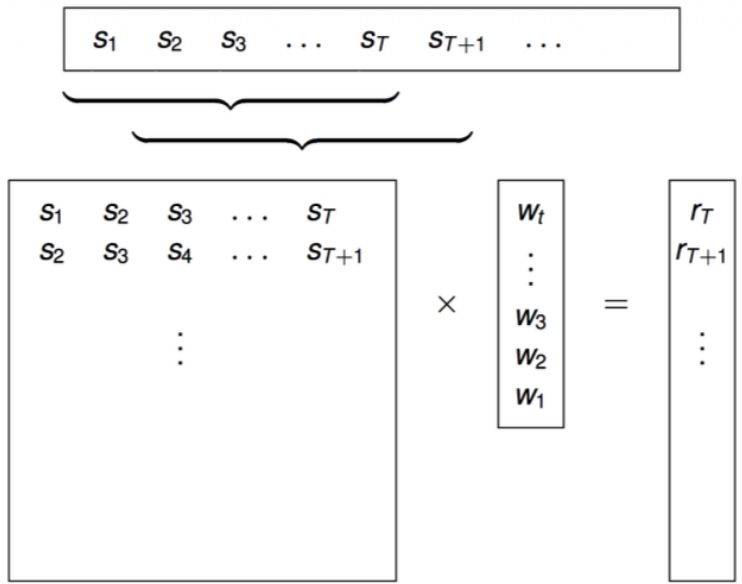
$$\hat{\mathbf{r}} = S\mathbf{w}$$

where $\mathbf{w}$ contains the filter coefficients.

We can solve this with linear regression!

$$\boxed{\hat{\mathbf{w}} = (S^\top S)^{-1} S^\top \mathbf{r}}$$

The STA therefore gives us a linear filter for a **deterministic encoding model**: $s \to \hat{r}(= \hat{x})$.

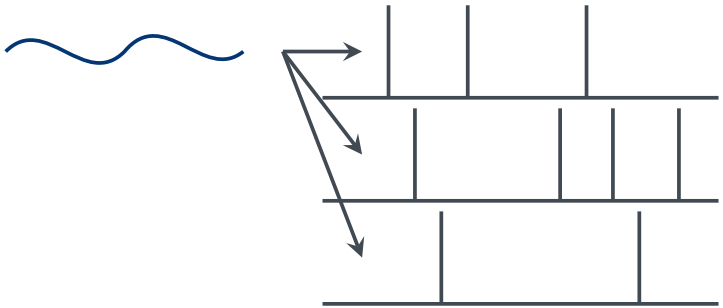



*Aside: For Gaussian white-noise stimuli, the STA recovers the linear filter $\mathbf{w}$ up to scale (Bussgang's Theorem).*

# However, neurons don't respond deterministically: Probabilistic $S \rightarrow X (\rightarrow R)$



**repeats of same stimulus $s$**

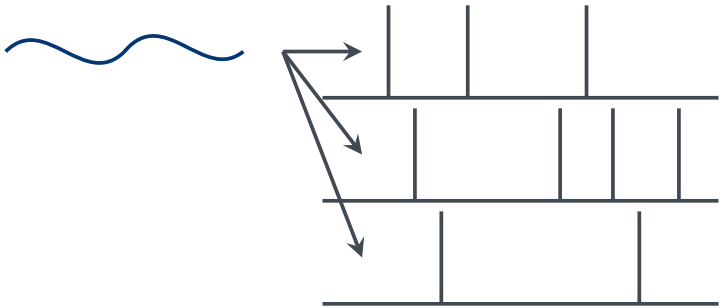


**$\rightarrow$ different spike trains**

# However, neurons don't respond deterministically: Probabilistic $S \rightarrow X(\rightarrow R)$



**repeats of same stimulus $s$**



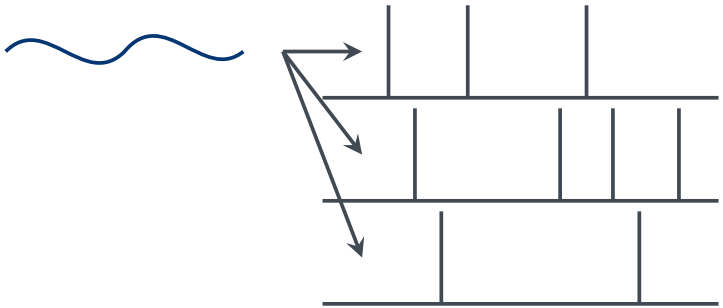
**$\rightarrow$ different spike trains**

+ limitations: overfitting, negative firing rates, linear baked-in

# However, neurons don't respond deterministically: Probabilistic $S \rightarrow X (\rightarrow R)$



**repeats of same stimulus $s$**



$\rightarrow$ **different spike trains**

$+$ limitations: overfitting, negative firing rates, linear baked-in

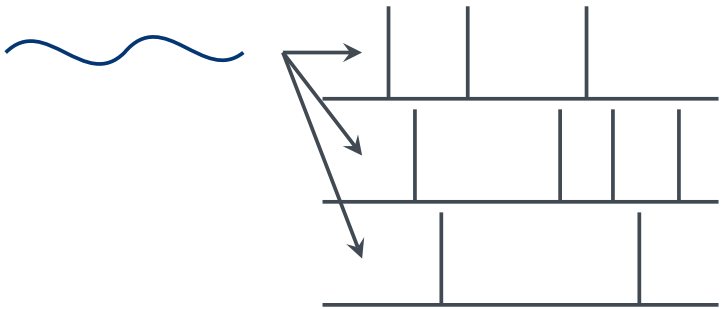
The coding problem asks how to infer the relationship between external variables and neural activity: $P(r \mid s)$.

# However, neurons don't respond deterministically: Probabilistic $S \rightarrow X (\rightarrow R)$



WITS UNIVERSITY

**repeats of same stimulus** $s$



$\rightarrow$ **different spike trains**

$+$ limitations: overfitting, negative firing rates, linear baked-in

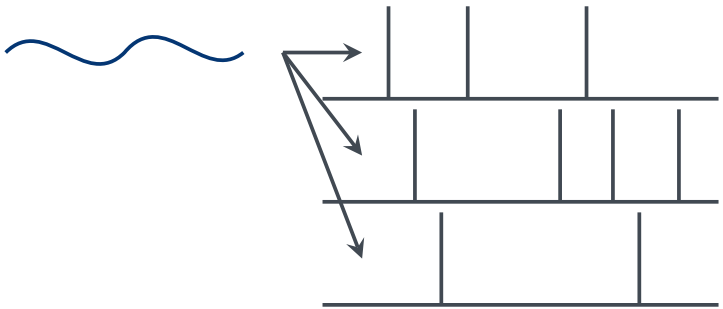
The coding problem asks how to infer the relationship between external variables and neural activity: $P(r \mid s)$.

We do this by finding an approximating model with parameters $\theta$: $P(r \mid s) \approx P_\theta(r \mid s)$.

# However, neurons don't respond deterministically: Probabilistic $S \to X (\to R)$



**repeats of same stimulus** $s$



$\to$ **different spike trains**

+ limitations: overfitting, negative firing rates, linear baked-in

The coding problem asks how to infer the relationship between external variables and neural activity: $P(r \mid s)$.

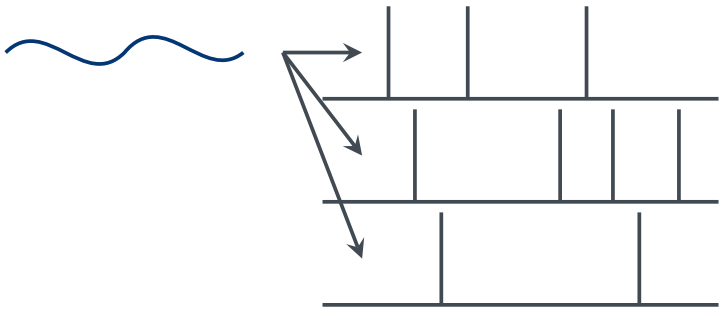
We do this by finding an approximating model with parameters $\theta$: $P(r \mid s) \approx P_\theta(r \mid s)$.

Model choice trades off ease of fit (e.g. linear / Gaussian) with richness and flexibility (e.g. multi-compartment Hodgkin-Huxley). Generalised linear models (GLMs) fit nicely in the middle.

# However, neurons don't respond deterministically: Probabilistic $S \rightarrow X (\rightarrow R)$



**repeats of same stimulus** $s$



$\rightarrow$ **different spike trains**

+ limitations: overfitting, negative firing rates, linear baked-in

The coding problem asks how to infer the relationship between external variables and neural activity: $P(r \mid s)$.

We do this by finding an approximating model with parameters $\theta$: $P(r \mid s) \approx P_\theta(r \mid s)$.

Model choice trades off ease of fit (e.g. linear / Gaussian) with richness and flexibility (e.g. multi-compartment Hodgkin-Huxley). Generalised linear models (GLMs) fit nicely in the middle.

GLMs allow us to move from **maximum likelihood estimation (MLE)** to estimate parameters for a single neuron model ($P(r_1 \mid s, \theta)$) to multi-neuron models ($P(r_1, ..., r_n \mid s, \theta)$).

# MLE for a single linear Poisson neuron

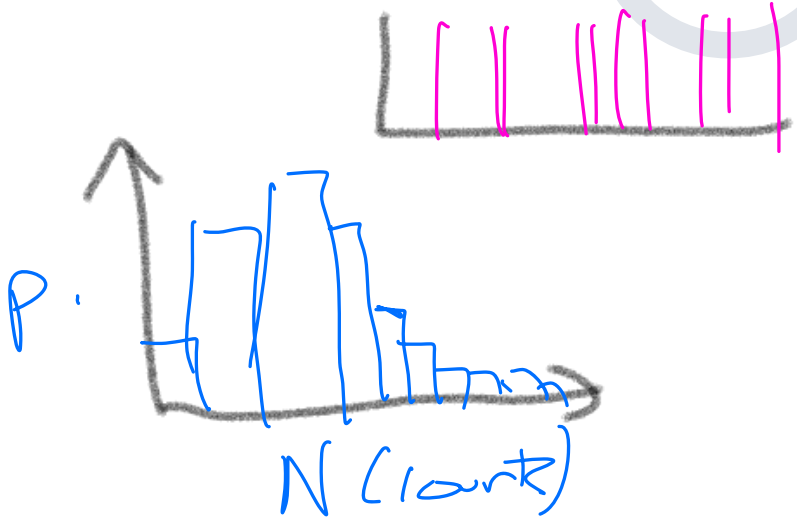


Suppose the stimulus is $s \geq 0 \in S$ is contrast and we observe $N$ spikes in a time window.

$$N \sim \text{Poisson}(\lambda),$$

where $\lambda = \theta s = f(s)$

is the neuron's firing rate (spike rate), with $\theta$ a parameter of the model. Why Poisson?



# MLE for a single linear Poisson neuron

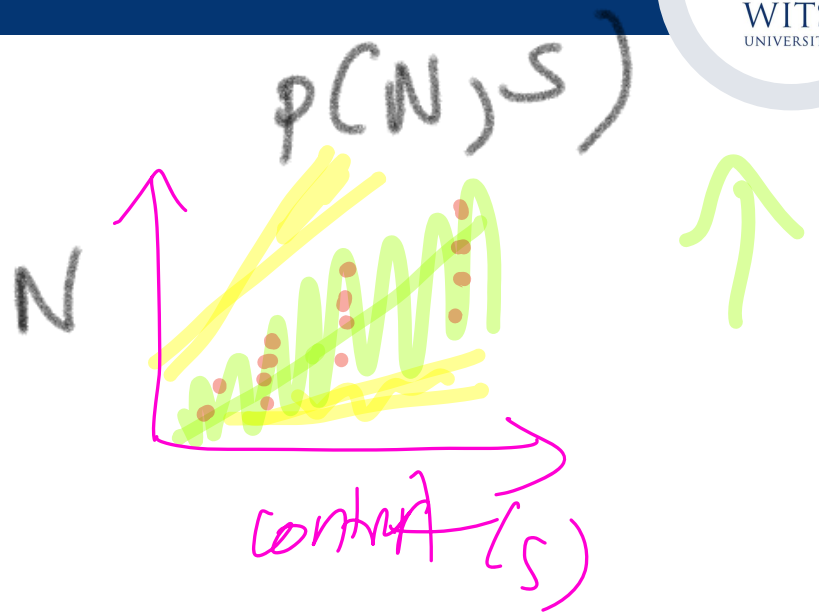


Suppose the stimulus is $s \geq 0 \in S$ is contrast and we observe $N$ spikes in a time window.

$$N \sim \text{Poisson}\,(\lambda)\,,$$

$$\text{where } \lambda = \theta s = f(s)$$

is the neuron's firing rate (spike rate), with $\theta$ a parameter of the model. Why Poisson?




The **encoding model** is

$$P(N \mid s, \theta) = \frac{1}{N!}(\lambda^N)e^{-\lambda}.$$

# Maximum likelihood estimation



Given observed data $(S, N)$, how can we find $\theta$ that maximises $P(N \mid S, \theta)$?

$$P(N \mid S, \theta) = \prod_{i=1} P(n_i \mid s_i, \theta)$$

*What assumption must we make here?*

# Maximum likelihood estimation



Given observed data $(S, N)$, how can we find $\theta$ that maximises $P(N \mid S, \theta)$?

$$P(N \mid S, \theta) = \prod_{i=1} P(n_i \mid s_i, \theta)$$

*What assumption must we make here?* $\rightarrow$ conditional independence between trials!

# Maximum likelihood estimation



Given observed data $(S, N)$, how can we find $\theta$ that maximises $P(N \mid S, \theta)$?

$$P(N \mid S, \theta) = \prod_{i=1} P(n_i \mid s_i, \theta) = \prod_{i=1} \frac{1}{n_i!} (\theta s_i)^{n_i} e^{-(\theta s_i)}$$

*What assumption must we make here?* → conditional independence between trials!

The **maximum-likelihood estimate** is

$$\hat{\theta}_{\mathrm{ML}} = \arg\max_{\theta} P(N \mid S, \theta).$$

$$\frac{d}{d\theta} P = 0$$

$$= \arg\max_{\theta} \log \left( \prod p(n_i \mid s_i, \theta) \right)$$

$$= \arg\max_{\theta} \sum_i \left[ n_i \log(\theta) - \theta s_i \right] + \text{constant}$$

$$= \log \theta (\Sigma n_i) - \theta (\Sigma s_i)$$

$$\frac{d}{d\theta} = 0 = \frac{1}{\theta} (\Sigma n_i) - (\Sigma s_i) \Rightarrow \theta = \frac{\Sigma n_i}{\Sigma s_i}$$

**Exercise:** Let's solve it!

# Maximum likelihood estimation



Given observed data $(S, N)$, how can we find $\theta$ that maximises $P(N \mid S, \theta)$?

$$P(N \mid S, \theta) = \prod_{i=1} P(n_i \mid s_i, \theta)$$

*What assumption must we make here?* $\rightarrow$ conditional independence between trials!

The **maximum-likelihood estimate** is

$$\boxed{\hat{\theta}_{\mathrm{ML}} = \arg\max_{\theta} P(N \mid S, \theta).}$$

**Exercise:** Let's solve it!

$$\hat{\theta}_{\mathrm{ML}} = \frac{\sum n_i}{\sum s_i}$$

The MLE is *consistent and efficient!*

# MLE for a single linear Gaussian neuron

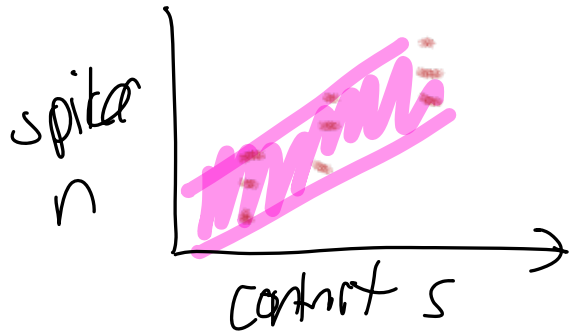


Suppose the stimulus is $s \in S$ and we observe $r$ rate.

$$r \sim N\left(\mu, \sigma^2\right),$$

$$\text{where } \mu = \theta s = f(s)$$

is the neuron's firing rate (spike rate), with $\theta$ a parameter of the model.



# MLE for a single linear Gaussian neuron



Suppose the stimulus is $s \in S$ and we observe $r$ rate.

$$r \sim N\left(\mu, \sigma^2\right),$$

$$\text{where } \mu = \theta s = f(s)$$

is the neuron's firing rate (spike rate), with $\theta$ a parameter of the model.

The **encoding model** is

$$P(r \mid s, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(r-\mu)^2}{2\sigma^2}\right].$$

# MLE for a single linear Gaussian neuron



Suppose the stimulus is $s \in S$ and we observe $r$ rate.

$$r \sim N\left(\mu, \sigma^2\right),$$

$$\text{where } \mu = \theta s = f(s)$$

is the neuron's firing rate (spike rate), with $\theta$ a parameter of the model.

The **encoding model** is

$$P(r \mid s, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(r-\mu)^2}{2\sigma^2}\right].$$

Solving in the same way,

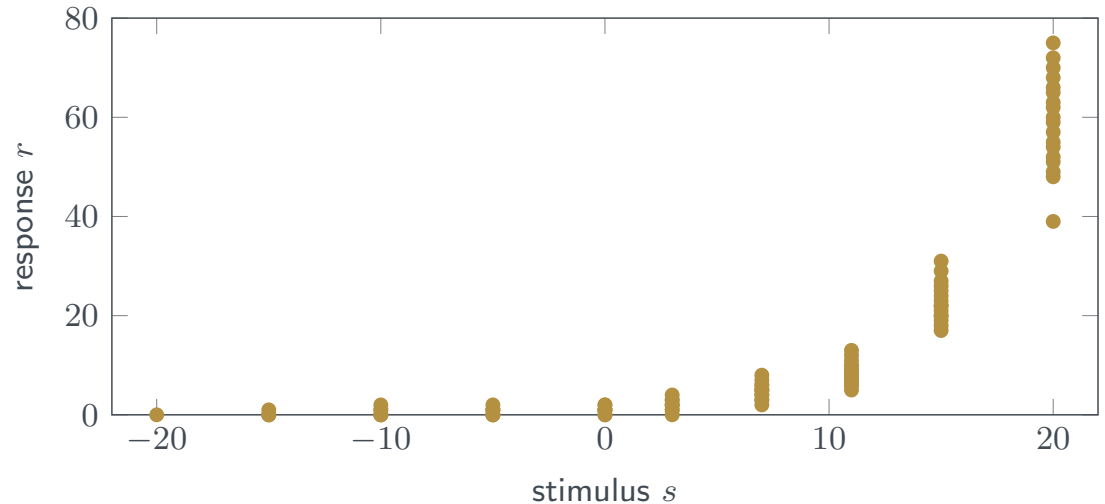
$$\hat{\theta}_{\mathrm{ML}} = \frac{\sum r_i s_i}{\sum s_i^2}$$

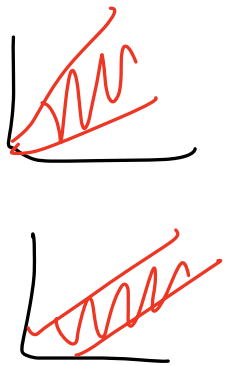
Does this look familiar? Why does it re-appear here?

# What about this neuron?



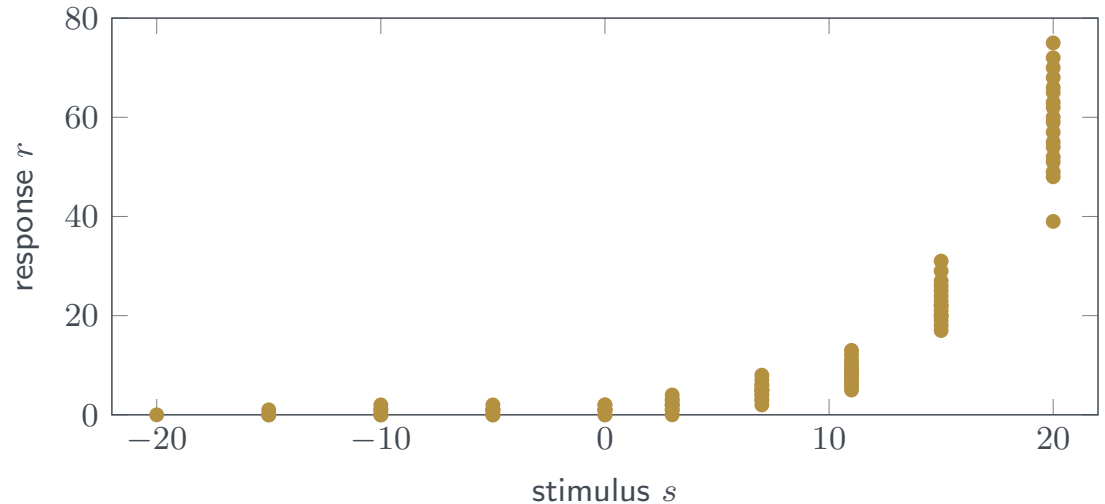
spike counts



# What about this neuron?





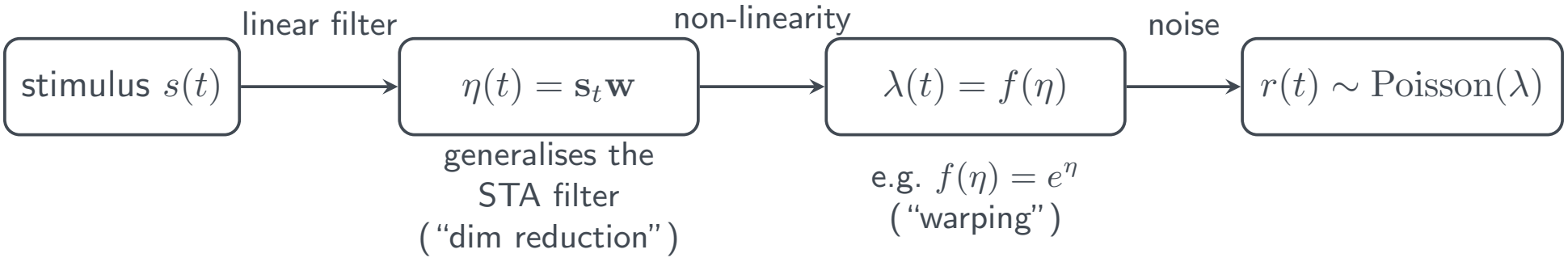

This needs a more general set-up: $r \sim \text{Poisson}(\lambda)$ but

$$\lambda = f(\theta^\top s)$$

where $f$ is a non-linearity (e.g. exponential). This is a **GLM!**

# A general model of neural responses






e.g.

$$\boxed{\lambda(t) = \exp(\mathbf{s}_t\mathbf{w})} \quad \Longrightarrow \quad r(t) \sim \text{Poisson}(\lambda(t))$$

Gaussian: $\vec{r} = f(\theta, s) + \varepsilon$ → $\mathcal{N}(0, \Theta)$.

Poisson: $r \sim \text{Poisson}(f(\theta, s))$

## The GLM

A stimulus is transformed into a firing rate through a linear filter and nonlinear function, and the observed neural response is probabilistic.

# Spike trains in the GLM: connection to STA



Spike trains can be modelled with GLMs in the same way. At time $t$:

$$r_t = \mathbf{s_t} \cdot \mathbf{w} + \text{noise}$$

If noise is assumed to be **Gaussian**, we recover the matrix version that can be solved as a least squares regression problem:



$$R = S\mathbf{w} + \text{noise}$$

# Spike trains in the GLM: connection to STA



Spike trains can be modelled with GLMs in the same way. At time $t$:

$$r_t = \mathbf{s_t} \cdot \mathbf{w} + \text{noise}$$

If noise is assumed to be **Gaussian**, we recover the matrix version that can be solved as a least squares regression problem:

$$R = S\mathbf{w} + \text{noise}$$

However, this noise assumption is **not valid** for *binary* spikes! So we might introduce a Bernoulli non-linearity...

# Spike trains in the GLM: connection to STA



Spike trains can be modelled with GLMs in the same way. At time $t$:

$$r_t = \mathbf{s_t} \cdot \mathbf{w} + \text{noise}$$

If noise is assumed to be **Gaussian**, we recover the matrix version that can be solved as a least squares regression problem:

$$R = S\mathbf{w} + \text{noise}$$

However, this noise assumption is **not valid** for *binary* spikes! So we might introduce a Bernoulli non-linearity...

Or a Poisson non-linearity to account for *spike counts*... All creating GLMs!

# GLM: Summary



1. **"Linear-Gaussian" GLM:**

$$r_t \mid \mathbf{s}_t \sim \mathcal{N}\left(\mathbf{s}_t \cdot \mathbf{w}, \sigma^2\right)$$

$$\hat{\mathbf{w}} = (S^\top S)^{-1} S^\top R$$

2. **Bernoulli GLM:**

$$r_t \mid \mathbf{s}_t \sim \mathrm{Ber}\left(f(\mathbf{s}_t \cdot \mathbf{w})\right)$$

$$\mathcal{L} = R^\top \log f(S\mathbf{w}) - (\mathbf{1} - R)^\top \log\left[1 - f(S\mathbf{w})\right]$$

3. **Poisson GLM:**

$$r_t \mid \mathbf{s}_t \sim \mathrm{Poiss}\left(\Delta f(\mathbf{s_t} \cdot \mathbf{w})\right)$$

$$\mathcal{L} = R^\top \log f(S\mathbf{w}) - \Delta \mathbf{1}^\top f(S\mathbf{w})$$

Try write out the Python code for each!

# GLM: Some applications (not for examination)



- Coding of perceptual decisions (Park et al., *Nature Neuroscience*, 2014)
- Predicting courtship singing in male flies (Coen et al., *Nature*, 2014)
- Coding of position/movement (Hardcastle et al., *Nature Neuroscience*, 2017)
- Alike to a deep neural network (or generative model) through stacking linear-non-linear layers until an output Poisson transformation (Yamins et al., 2014; Benjamin et al., 2017)
- GLMs can be readily generalised to incorporate spike-history dependence (adaptation) and dependencies between multiple neurons (correlations)
- Many more!

# From a single neuron to a population of neurons



Instead of one neuron:

$$r_1, r_2, \ldots, r_N$$

Define the population response

$$\boxed{\boldsymbol{r} = (r_1, \ldots, r_N)}$$

Each neuron has a different tuning curve:

$$r_i = f_i(\theta)$$

Together:

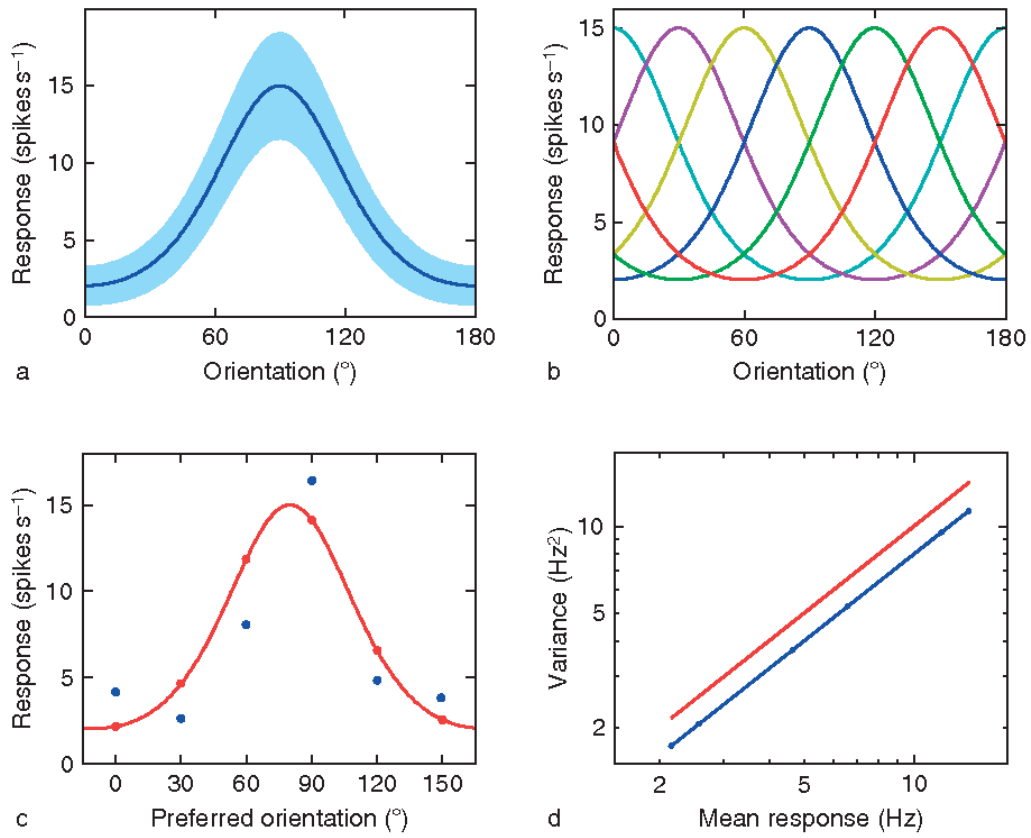
$$\theta \quad \longrightarrow \quad \boldsymbol{r}(\theta)$$

## Population coding

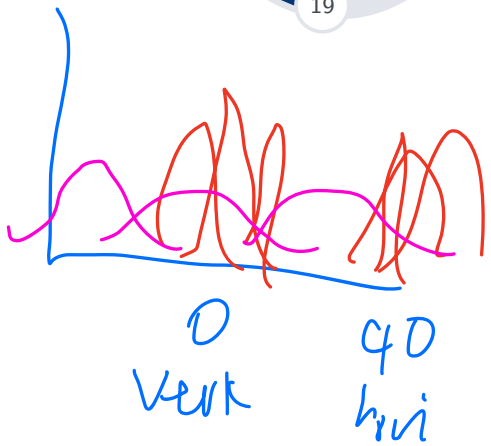
A stimulus (distribution) can be represented by a **pattern of activity** across many neurons.

# From a single neuron to a population of neurons






ple of encoding a stimulus in a population code for orientation. (a) Response of a single neuron as a fu




Ma and Pouget, 2009

# From a single neuron to a population of neurons



Population responses can also be understood via a GLM:

$$\mathbf{s}_t \longrightarrow \begin{bmatrix} \lambda_{1,t} \\ \lambda_{2,t} \\ \vdots \\ \lambda_{N,t} \end{bmatrix} \longrightarrow \begin{bmatrix} r_{1,t} \\ r_{2,t} \\ \vdots \\ r_{N,t} \end{bmatrix}$$

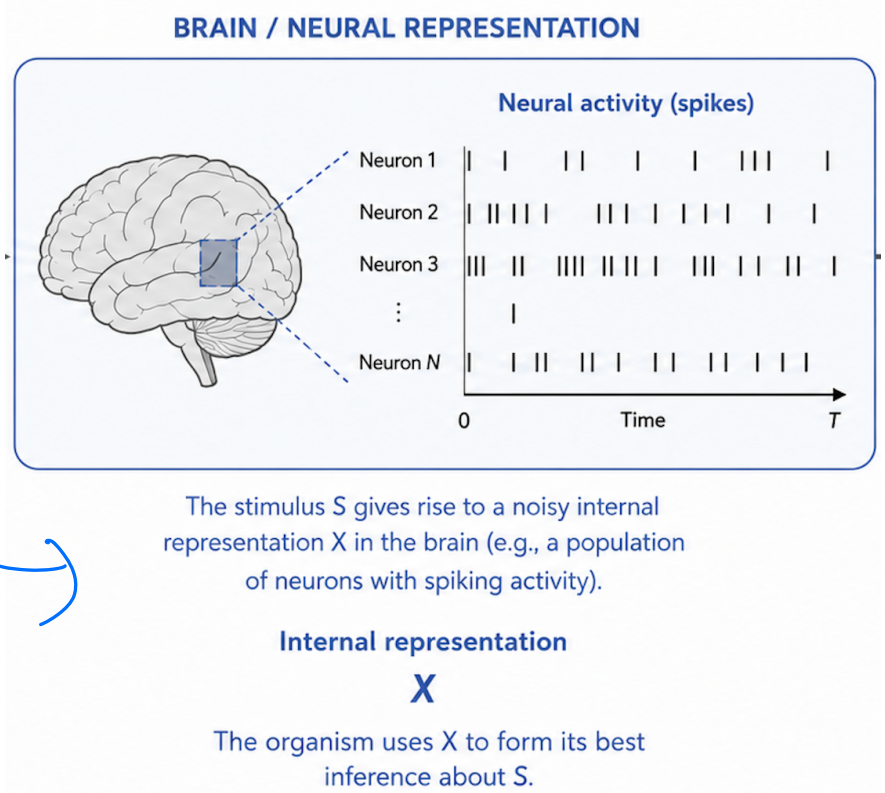
$$\underbrace{\mathbf{s}_t}_{\text{stimulus}} \longrightarrow \underbrace{\boldsymbol{\lambda}_t}_{\text{population firing rates}} \longrightarrow \underbrace{\mathbf{r}_t}_{\text{population response}}$$

where

$$\lambda_{i,t} = f_i\left(\mathbf{s}_t^\top \mathbf{w}_i\right), \qquad i = 1, \ldots, N$$

# Latent variables and latent variable models






We assume that the organism doing our experiment cares about our stimuli $S$ — but they could be:

- Looking out the window;
- Using a different clue to solve the task;
- Biased by an internal belief state;
- Representing motor activity in that part of cortex;
- Integrating a high-dimensional cue.

How do we know what the salient (latent) variables are?

# Latent variables and latent variable models



One option is to use **latent variable models** without assuming a given $S$. These are based on the Bayesian computation

$$p(x, s) = p(x \mid s)p(s)$$

to find the latent variable distribution $p(s)$.

Usually these are low-dimensional or discrete states, and can be found unsupervised by assuming some properties of the latent distribution and a mapping.

We will not go in depth here, but mention a few examples and the principle via PCA.

Examples: factor analysis (PCA is a special case); VAEs; latent dynamical models like the Kalman filter; ...

# Population code geometry and latent variables



In population codes, each stimulus corresponds to a point in an $N$-dimensional neural state space.

$$\theta \quad \longrightarrow \quad \boldsymbol{r}(\theta) \in \mathbb{R}^N$$

Similar stimuli $\rightarrow$ nearby population states

Now imagine recording from 1000 neurons. At every time point:

$$\boldsymbol{r}(t) = (r_1(t), \ldots, r_{1000}(t)) \in \mathbb{R}^{1000}$$

## Problem

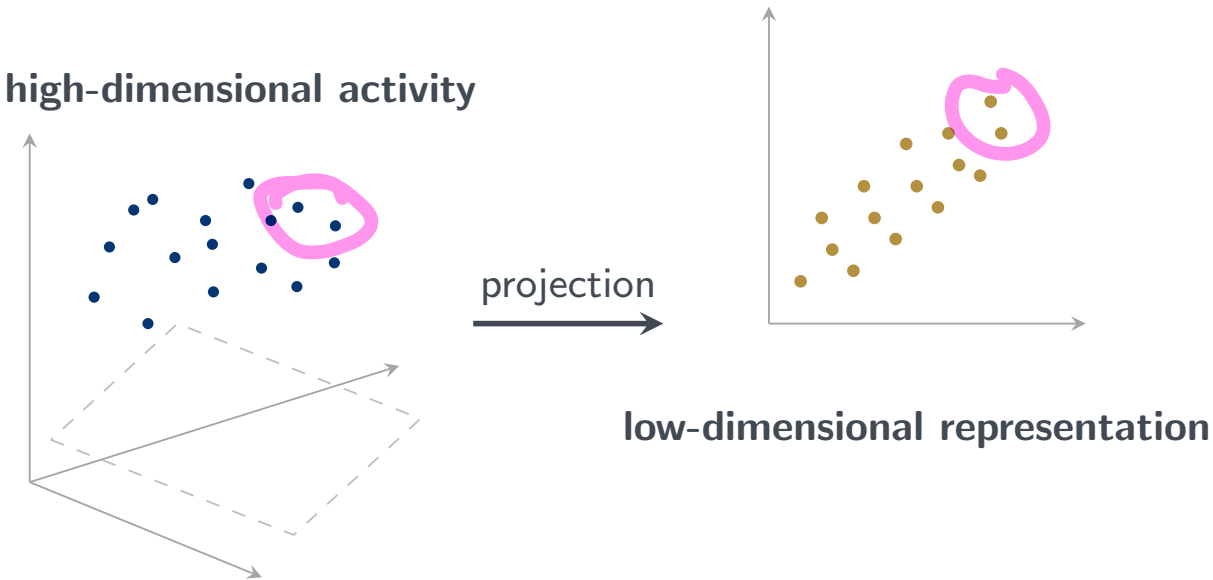
How do we visualise a 1,000-dimensional neural population?

# Dimensionality reduction



Look for a smaller set of variables that captures important structure in the population activity:

$$\boldsymbol{r}(t) \in \mathbb{R}^N \qquad \text{with} \qquad \boldsymbol{z}(t) \in \mathbb{R}^K, \quad K \ll N$$



# e.g. Principal Component Analysis



Given a population activity matrix

$$X \in \mathbb{R}^{T \times N},$$

PCA finds directions of maximal variance.

$$X \approx ZW^\top$$

where

$$Z \in \mathbb{R}^{T \times K}, \qquad K \ll N.$$

**Question: what does the low-dimensional structure mean?**

# Latent variables from PCA



One way to think about population activity:

latent state $\longrightarrow$ neural population

$$\boldsymbol{r}_t = W\boldsymbol{z}_t + \boldsymbol{\epsilon}_t$$

where

- $\boldsymbol{z}_t$ = latent variables
- $W$ = mapping to neurons
- $\boldsymbol{\epsilon}_t$ = noise

# Primer: Where does the population activity come from?



So far we have treated

$$\boldsymbol{r}(t)$$

almost as if it simply **appeared**. But as you know from previous lectures, neurons are connected!

$$\tau \frac{d\boldsymbol{r}}{dt} = -\boldsymbol{r} + F(W\boldsymbol{r} + \boldsymbol{I})$$

## Next: How does coding arise from dynamics?

What does the recurrent connectivity $W$ do to the dynamics of the population? What is the role of inhibition?

# Take home and resources



| Question | Description | Tool |
|---|---|---|
| What does a neuron respond to? | Tuning | Tuning curve / STA |
| How does a stimulus generate spikes? | Neural coding $P(r \mid s)$ | MLE $\rightarrow$ GLMs |
| What structure exists in population activity? | Latent state $\mathbf{z}$ | PCA / latent models |

GLM tutorials:

- Neuromatch: https://compneuro.neuromatch.io/tutorials/W1D3_GeneralizedLinearModels/student/W1D3_Intro
- Jonathan Pillow's lab: https://github.com/pillowlab/GLMspiketraintutorial_python